Université Mundiapolis
Cloud computing

Conteneuriser un modèle iris, puis compter ses prédictions avec Redis

Houda Erimi
15 octobre 2026

Introduction

Ce travail se fait en deux temps. D'abord je mets un petit modèle de classification dans Docker : un conteneur l'entraîne, un autre l'utilise. Ensuite, pour le TP 4, je transforme cette prédiction en API et j'ajoute Redis. Redis ne sert pas à deviner la fleur. Il compte seulement combien de fois l'API a été appelée, et il se souvient de ce nombre tant que son volume existe.

Le modèle travaille sur le jeu iris : 150 fleurs, 4 mesures en centimètres, 3 espèces. Quand je ne donne pas de mesures, l'exemple utilisé est une setosa, `5.1, 3.5, 1.4, 0.2`.

1 Mettre le modèle dans Docker

1.1 Deux conteneurs, un volume

L'entraînement est dans `train/`. Le script apprend une forêt aléatoire et écrit `iris_model.joblib` dans `/model`. Ce dossier est un volume : le fichier reste après l'arrêt du conteneur.

L'inférence est dans `infer/`. Elle ne réapprend rien. Elle ouvre le fichier déjà écrit et affiche l'espèce. Les deux services du premier Compose partagent le même volume, et l'inférence attend que l'entraînement soit terminé.

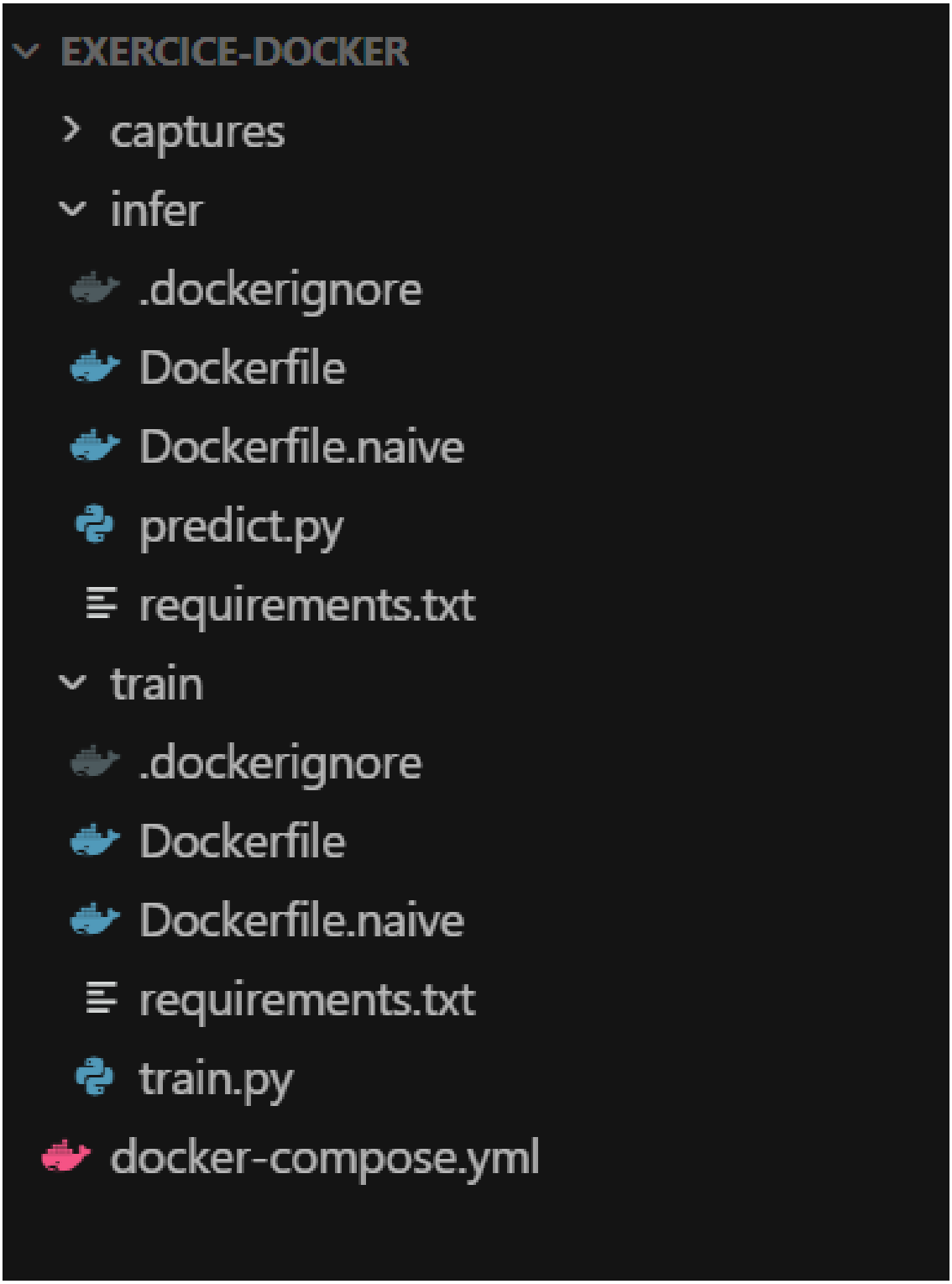


FIGURE 1 – Le dépôt au départ : entraînement, inférence, et le Compose qui les enchaîne.

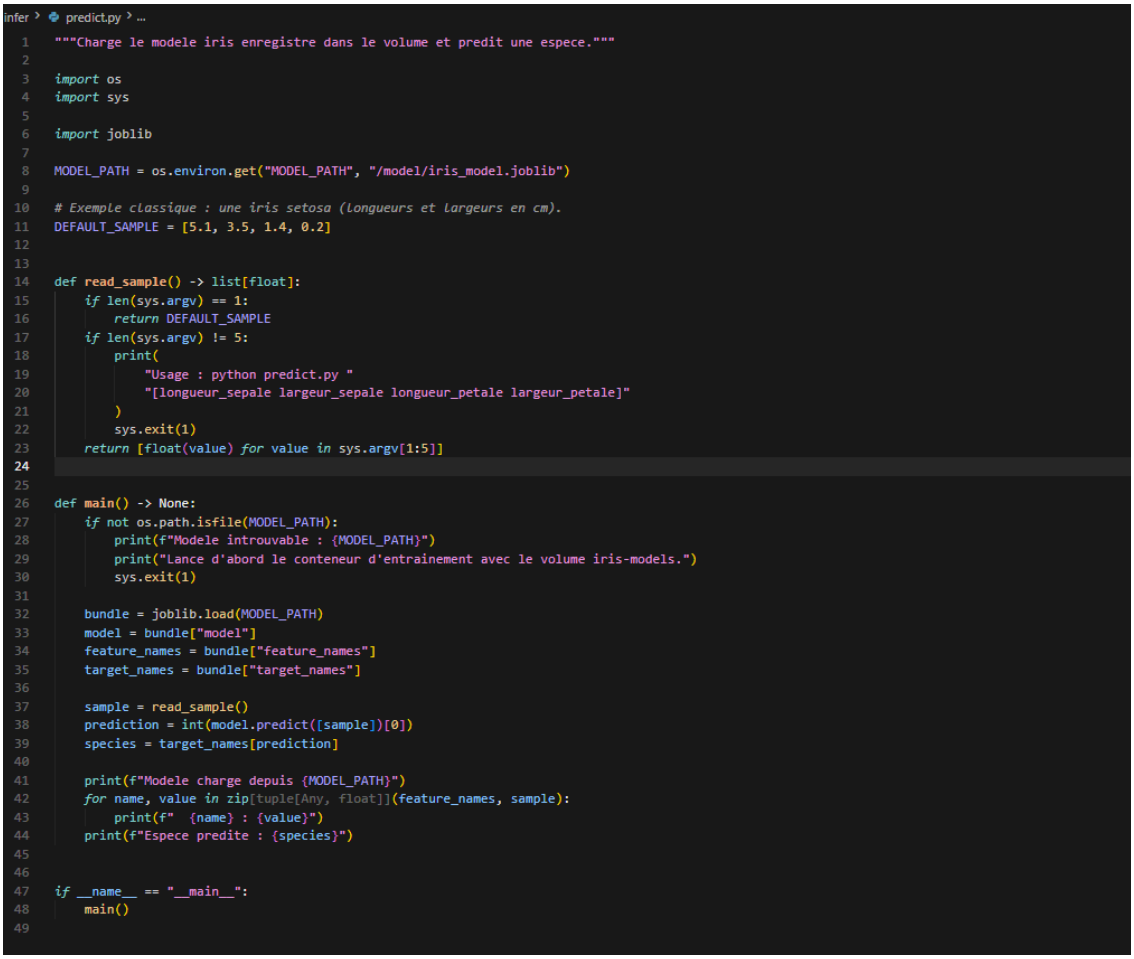


FIGURE 2 – Le script de prédiction. Il charge le modèle dans le volume et affiche l'espèce.

1.2 La version naïve, puis la version allégée

La première image part de `python:3.11`, copie les fichiers, puis installe les bibliothèques. Elle fonctionne, mais elle est lourde : presque toute la place est prise par l'image Python complète.

Je l'ai ensuite allégée de quatre façons.

— La base devient `python:3.11-slim`.

— Le build a deux étapes. La première installe les paquets, la seconde ne garde que le résultat.

— `requirements.txt` est copié et installé avant le script Python. Un changement de code ne relance plus `pip`.

— Un `.dockerignore` laisse de côté les caches et les fichiers inutiles.

L'image finale a quand même besoin de `libgomp1`, sinon scikit-learn ne démarre pas.

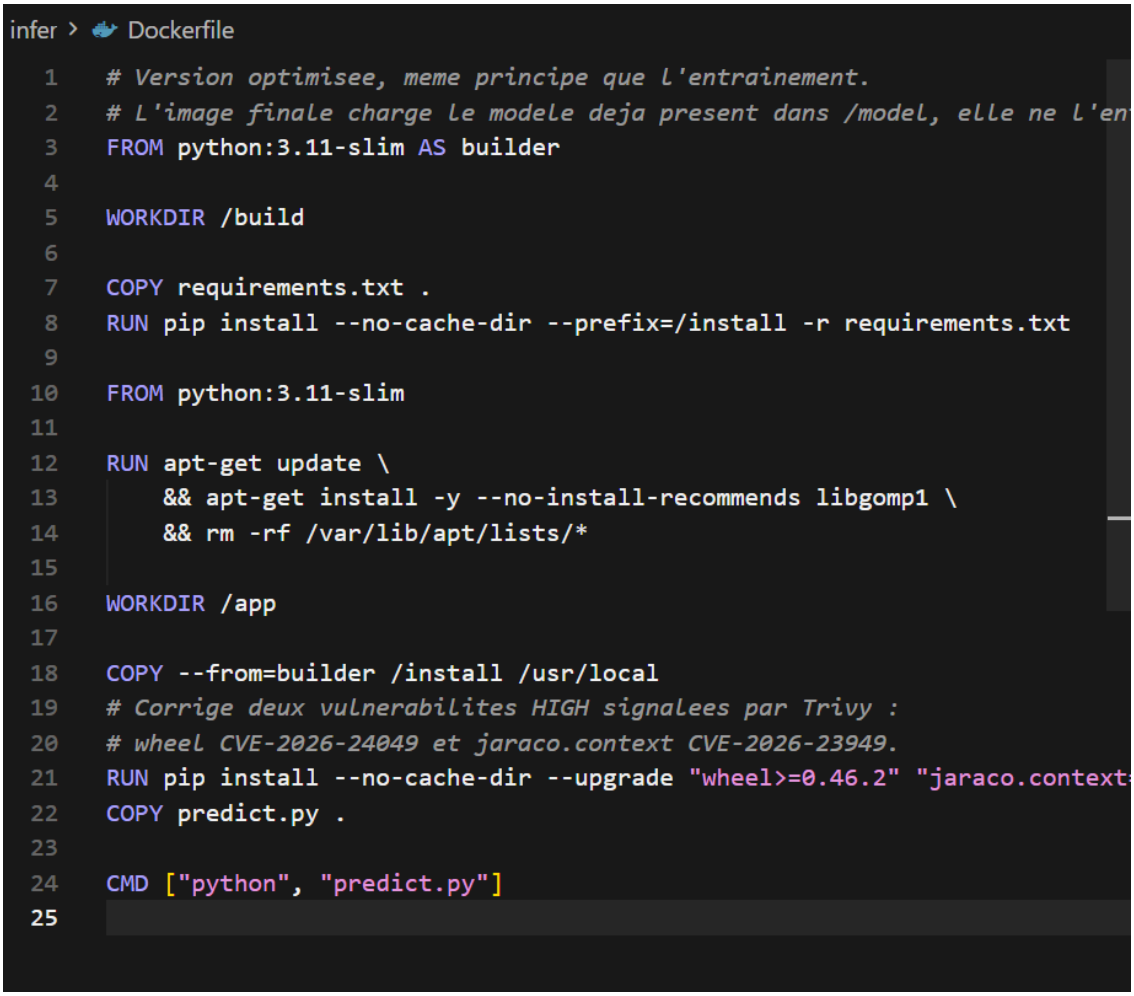


FIGURE 3 – Dockerfile d'inférence optimisé, avant d'en faire une API.

J'ai mesuré les deux versions sur la même machine.

Version	Taille	Build à froid	Build à chaud
Naïve, `python:3.11`	1,98 Go	53 s et 76 s	environ 2 s
Optimisée, `python:3.11-slim`	559 Mo	65 s et 62 s	environ 2 s

Les deux durées à froid sont l'entraînement puis l'inférence. L'image optimisée est environ 3,5 fois plus petite. Le premier build ne devient pas vraiment plus court, parce qu'il faut encore télécharger NumPy, SciPy et scikit-learn. Le gain, c'est la taille. Une fois les couches en cache, les deux versions se reconstruisent en deux secondes.

1.3 Ce que Trivy a trouvé

J'ai scanné l'image optimisée avec Trivy. Il n'y avait aucune faille critique. En revanche, deux failles hautes avaient déjà un correctif : `wheel` 0.45.1 (CVE-2026-24049) et `jaraco.context` 5.3.0 (CVE-2026-23949). Elles ne venaient pas de mon `requirements.txt`. Elles étaient embarquées dans `setuptools`, y compris dans des copies internes du paquet.

J'ai donc mis à jour les deux paquets, et `setuptools` lui-même en 84.0.0, sinon les vieilles copies restaient visibles pour le scan. Après reconstruction, Trivy ne signale plus de faille haute ni critique sur les paquets Python.

2 TP 4 : l'API et Redis

2.1 Ce que renvoie l'API

C'est le point que j'ai mis du temps à séparer. Une réponse de `/predict` contient deux choses qui n'ont pas le même rôle.

Champ	Rôle
`mesures`	Les quatre tailles envoyées, ou l'exemple setosa.
`espece`	La prédiction. Le modèle choisit l'espèce. Ici, `setosa`.
`predictions`	Le compteur. Ce n'est pas une espèce. C'est le nombre d'appels à `/predict`.

`/stats` ne calcule rien. Il redemande à Redis la valeur du compteur. Une réponse `{"predictions":0}` ne veut donc pas dire que le modèle s'est trompé. Elle veut dire qu'aucune prédiction n'a encore été demandée depuis le dernier effacement de Redis.

2.2 Le code et le Compose

`infer/api.py` charge le même fichier modèle. À chaque `GET /predict`, il devine l'espèce puis incrémente la clé `predictions` dans Redis. Le service Redis s'appelle `base`, comme demandé dans l'énoncé.

infer › api.py › ...

```python
"""API iris : predit une espece et compte chaque prediction dans Redis."""

import os

import joblib
import redis
from fastapi import FastAPI, Query

MODEL_PATH = os.environ.get("MODEL_PATH", "/model/iris_model.joblib")
REDIS_HOST = os.environ.get("REDIS_HOST", "base")
COUNTER_KEY = "predictions"

app = FastAPI(title="API iris")
store = redis.Redis(host=REDIS_HOST, port=6379, decode_responses=True)

_bundle = None


def load_bundle():
    global _bundle
    if _bundle is None:
        _bundle = joblib.load(MODEL_PATH)
    return _bundle


@app.get("/predict")
def predict(
    sepal_length: float = Query(5.1),
    sepal_width: float = Query(3.5),
    petal_length: float = Query(1.4),
    petal_width: float = Query(0.2),
):
    bundle = load_bundle()
    sample = [sepal_length, sepal_width, petal_length, petal_width]
    species = bundle["target_names"][int(bundle["model"].predict([sample])[0])]
    total = store.incr(COUNTER_KEY)
    return {"espece": species, "mesures": sample, "predictions": total}


@app.get("/stats")
def stats():
    total = store.get(COUNTER_KEY)
    return {"predictions": int(total or 0)}

```

FIGURE 4 – Les deux routes : prédire et compter, puis seulement relire le compteur.

Le fichier `docker-compose.yml` lance les deux services. `api` publie le port 8000 et monte le volume du modèle déjà entraîné. `base` est `redis:7-alpine`, et son dossier `/data` est le volume `redis-data`. C'est ce volume qui garde le compteur.

docker-compose.yml

```yaml
# TP 4 : l'API predit, Redis (service "base") compte les predictions.
# Le compteur est dans le volume redis-data : il survit a "down", pas a "down -v".

services:
  api:
    build: ./infer
    image: iris-api:tp4
    ports:
      - "8000:8000"
    environment:
      REDIS_HOST: base
      MODEL_PATH: /model/iris_model.joblib
    volumes:
      - iris-models:/model
    depends_on:
      - base

  base:
    image: redis:7-alpine
    volumes:
      - redis-data:/data

volumes:
  iris-models:
    external: true
    name: exercice-docker_iris-models
  redis-data:
```

FIGURE 5 – Compose du TP : `api` et `base`.

`docker compose config` ne démarre rien. Il réécrit la configuration telle que Compose la comprend. On y voit le port 8000, l'image Redis, et le volume externe du modèle.

```
PS C:\cours_cloudcomputing\exercice-docker> docker compose config
name: exercice-docker
services:
  api:
    build:
      context: C:\cours_cloudcomputing\exercice-docker\infer
      dockerfile: Dockerfile
    depends_on:
      base:
        condition: service_started
        required: true
    environment:
      MODEL_PATH: /model/iris_model.joblib
      REDIS_HOST: base
    image: iris-api:tp4
    networks:
      default: null
    ports:
      - mode: ingress
        target: 8000
        published: "8000"
        protocol: tcp
    volumes:
      - type: volume
        source: iris-models
        target: /model
        volume: {}
  base:
    image: redis:7-alpine
    networks:
      default: null
    volumes:
      - type: volume
        source: redis-data
        target: /data
        volume: {}
networks:
  default:
    name: exercice-docker_default
volumes:
  iris-models:
    name: exercice-docker_iris-models
    external: true
  redis-data:
    name: exercice-docker_redis-data
PS C:\cours_cloudcomputing\exercice-docker>
```

FIGURE 6 – Vérification avec `docker compose config`.

2.3 Le premier build a échoué

Au premier `docker compose up -build`, Redis s'est bien téléchargé. L'installation Python, non. Le pip 24.0 livré avec l'image a refusé un paquet téléchargé, en disant que son hash ne correspondait pas. Le build s'est arrêté là.

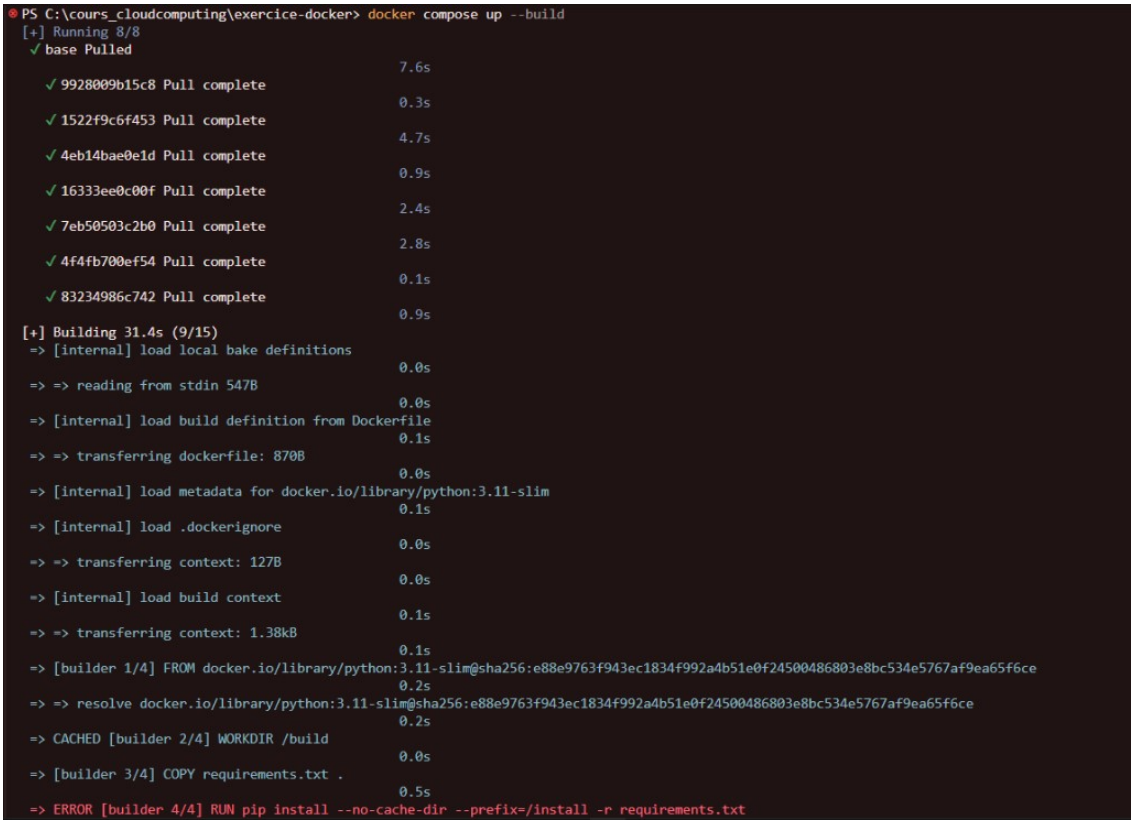


FIGURE 7 – Le build s'arrête sur `pip install`.

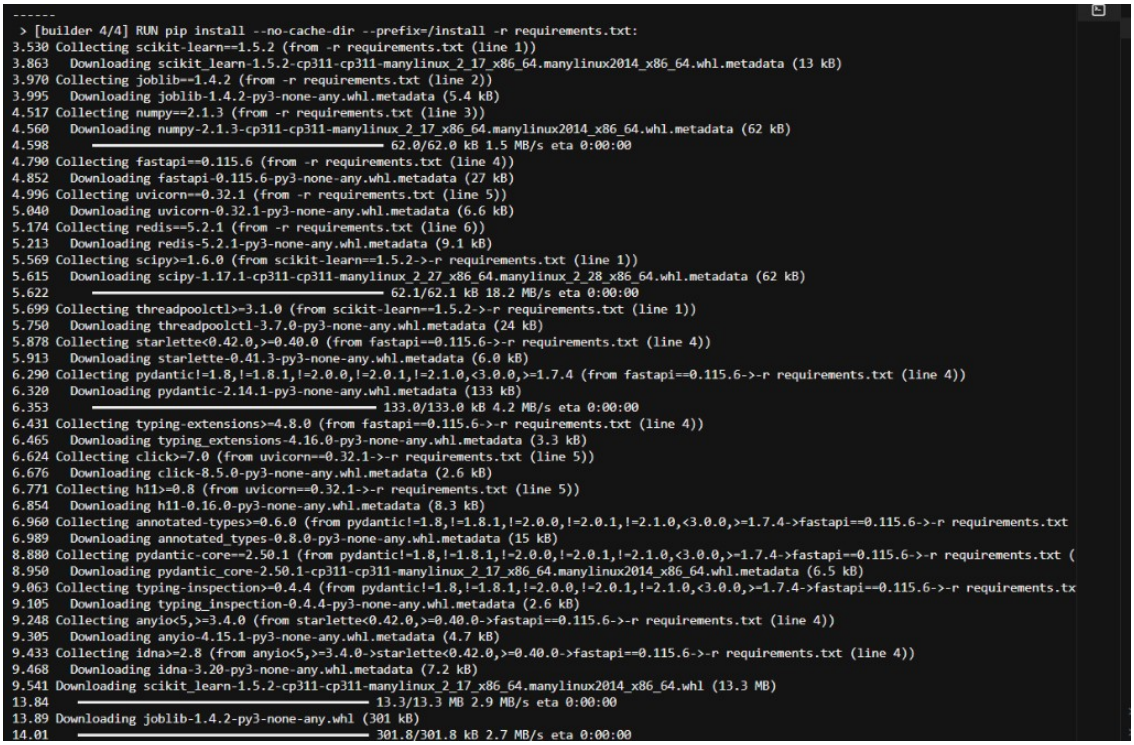


FIGURE 8 – Les paquets sont en train d'être téléchargés.

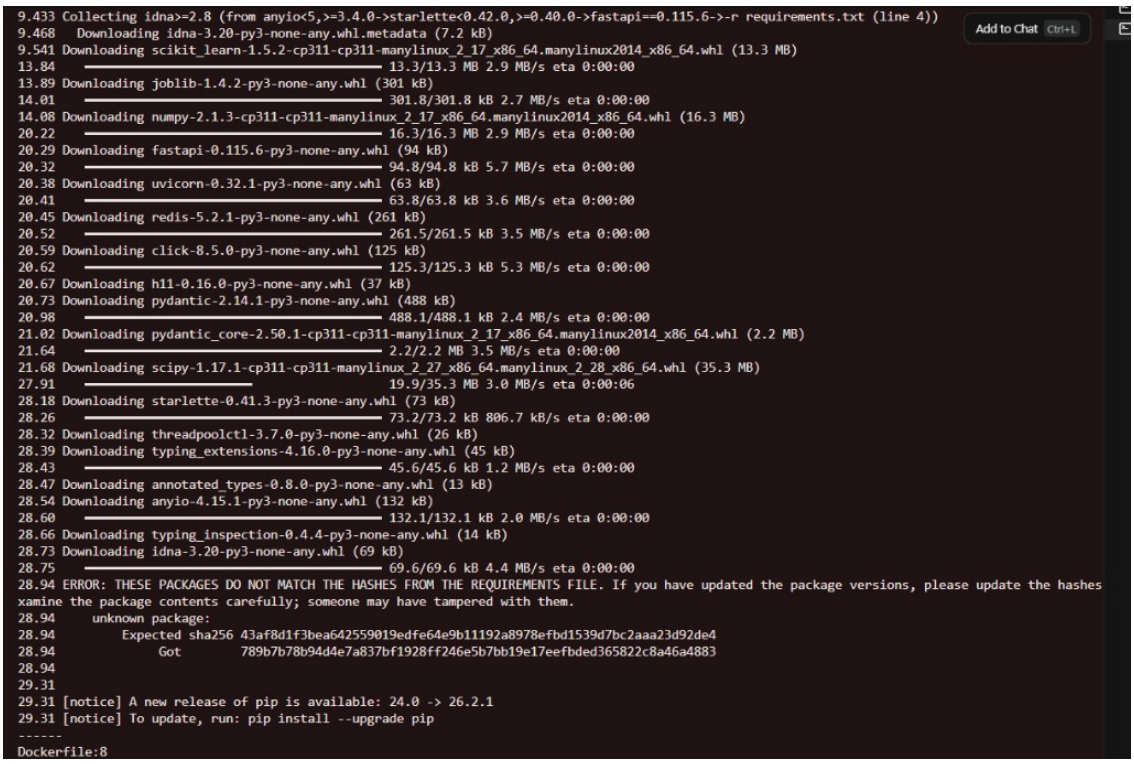


FIGURE 9 – Le message de hash. Le nom du paquet n'est même pas indiqué.

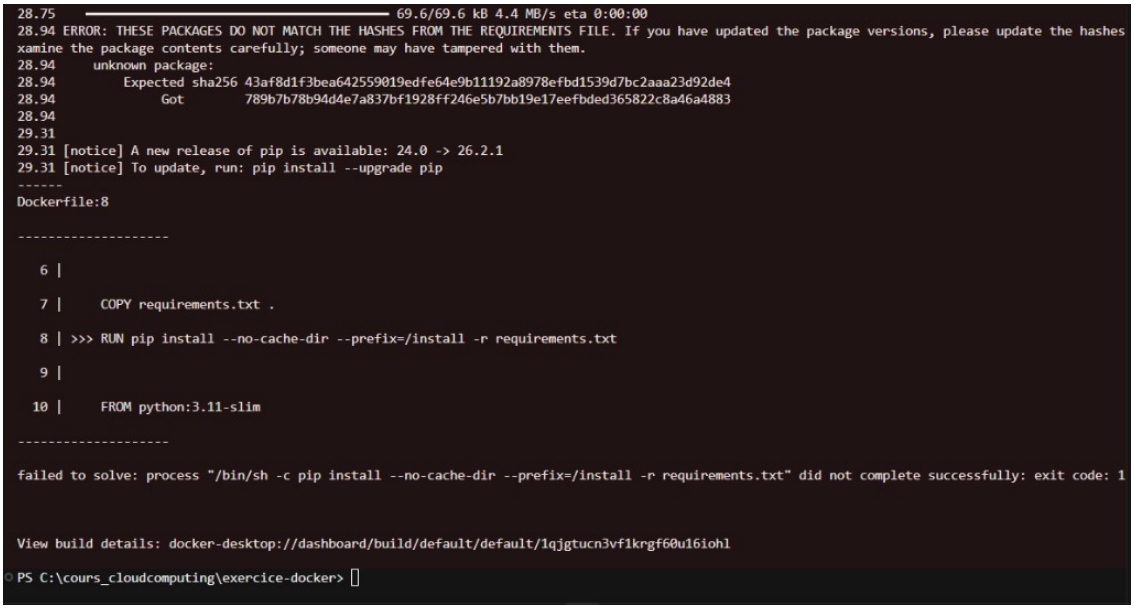


FIGURE 10 – Échec final du build, code 1.

J'ai modifié le Dockerfile pour mettre pip à jour avant d'installer `requirements.txt`. Avec pip 26.2.1, la même installation passe. Au deuxième lancement, les couches sont en cache, Redis annonce qu'il accepte les connexions, et Uvicorn écoute sur le port 8000.

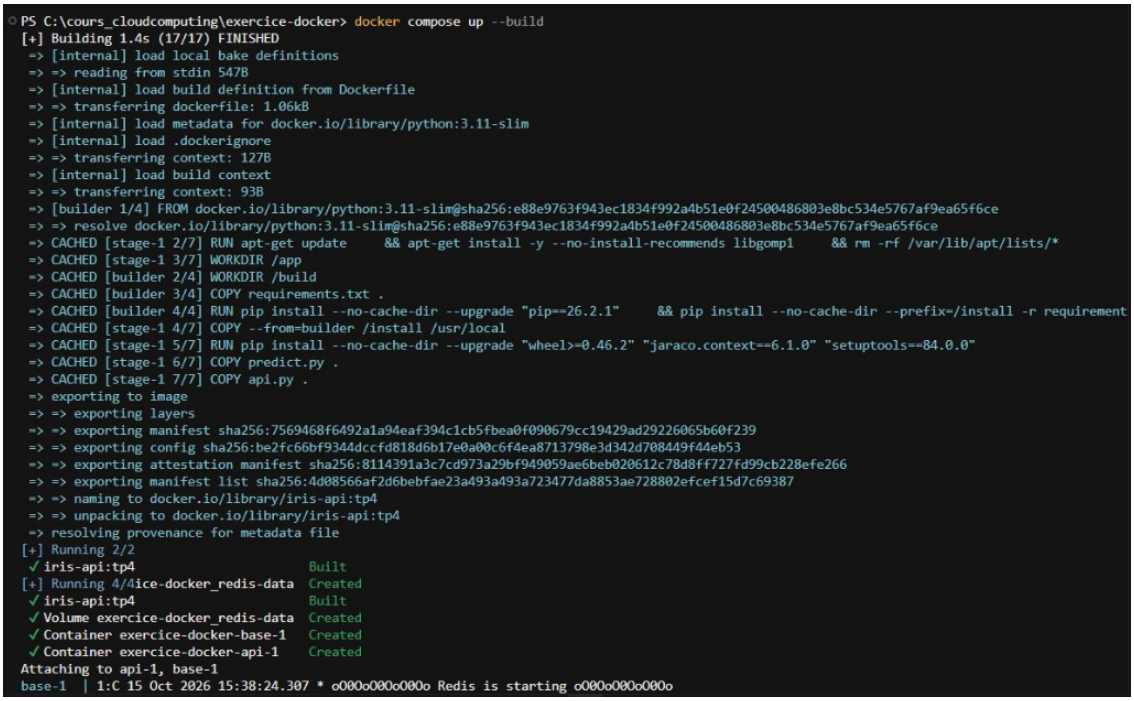


FIGURE 11 – Deuxième lancement : l'image est construite, Redis démarre.

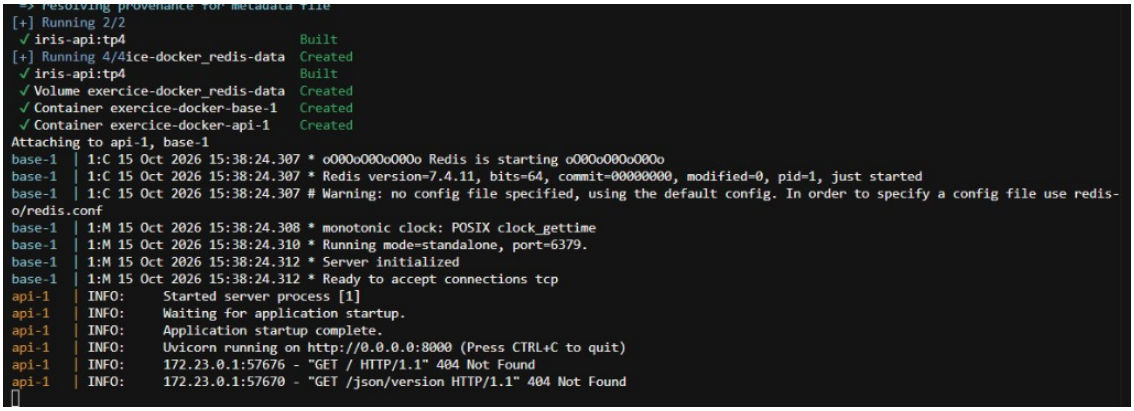


FIGURE 12 – L'API est prête. Les 404 viennent d'une sonde sur des adresses qui n'existent pas.

Les lignes `404` sur `/` et `/json/version` ne viennent pas du TP. Quelque chose interroge ces adresses tout seul. Les seules adresses utiles sont `/predict` et `/stats`.

2.4 Plusieurs prédictions

J'ai d'abord ouvert `http://localhost:8000/predict` dans Firefox. La page restait blanche, avec seulement la case « Pretty-print ». Pourtant le conteneur répondait `200`. Le JSON était bien renvoyé, le navigateur ne le dessinait pas.



FIGURE 13 – Firefox affiche une page vide alors que l'API a répondu.

Avec `curl.exe`, la réponse est lisible. Deux appels de suite donnent la même espèce, setosa, et le compteur passe de 3 à 4. `/stats` confirme `{"predictions":4}`.

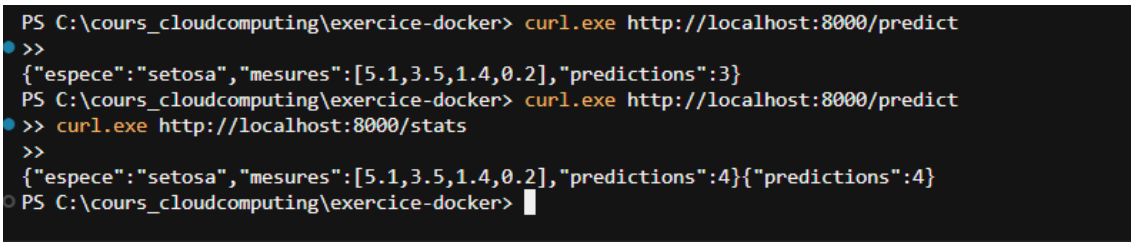


FIGURE 14 – Deux prédictions, puis le total lu dans Redis.

2.5 Le compteur et les volumes

J'ai arrêté la pile avec `docker compose down`, puis je l'ai relancée avec `docker compose up`. Je n'ai pas rappelé `/predict`. `/stats` valait encore 4.

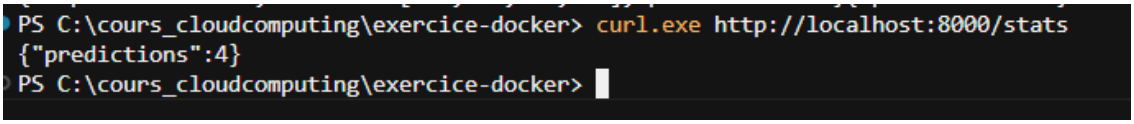


FIGURE 15 – Après un simple `down`, le compteur est toujours à 4.

Dans les logs de ce redémarrage, Redis écrit `Loading RDB` puis `keys loaded: 1`. Il a retrouvé sa sauvegarde sur le volume. `down` supprime les conteneurs, pas le volume.

J'ai ensuite fait `docker compose down -v`. Cette fois la sortie indique `Volume exercice-docker_redis-data Removed`. Au `up` suivant, Redis démarre sans recharger de clé. `/stats` vaut 0.

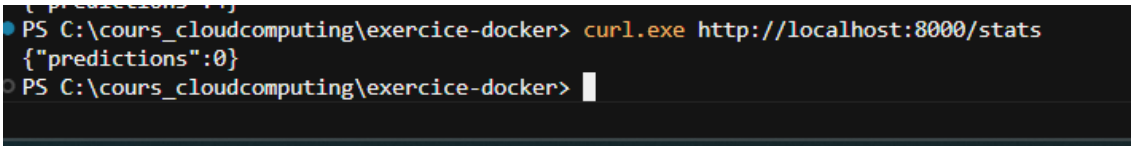


FIGURE 16 – Après `down -v`, le compteur repart de zéro.

C'est le résultat attendu des étapes 6 et 7. Le `0` concerne uniquement la mémoire du compteur. Le fichier du modèle est dans un autre volume, celui créé par l'entraînement, et il n'a pas été supprimé. Un nouvel appel à `/predict` répondrait encore `setosa`, et le compteur passerait à 1.

Conclusion

Le modèle iris tourne dans deux images Docker, d'abord dans une version simple de 1,98 Go, puis dans une version de 559 Mo. L'entraînement et l'inférence se partagent le fichier du modèle par un volume. Le scan n'a pas montré de faille critique ; les deux failles hautes corrigées étaient des paquets trop anciens dans l'image de base.

Le TP 4 ajoute une API et Redis. L'API prédit l'espèce. Redis compte les appels. Ce compteur survit à `docker compose down`, parce que le volume reste. Il disparaît avec `docker compose down -v`, parce que le volume est supprimé avec les conteneurs. C'est cette différence que je retiens pour la suite, quand le même couple `api` et `base` sera déployé sur Kubernetes.